Data Storage

Rashmi Dutta Baruah

Department of Computer Science & Engineering

भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Technology Guwahati
Guwahati - 781039, INDIA

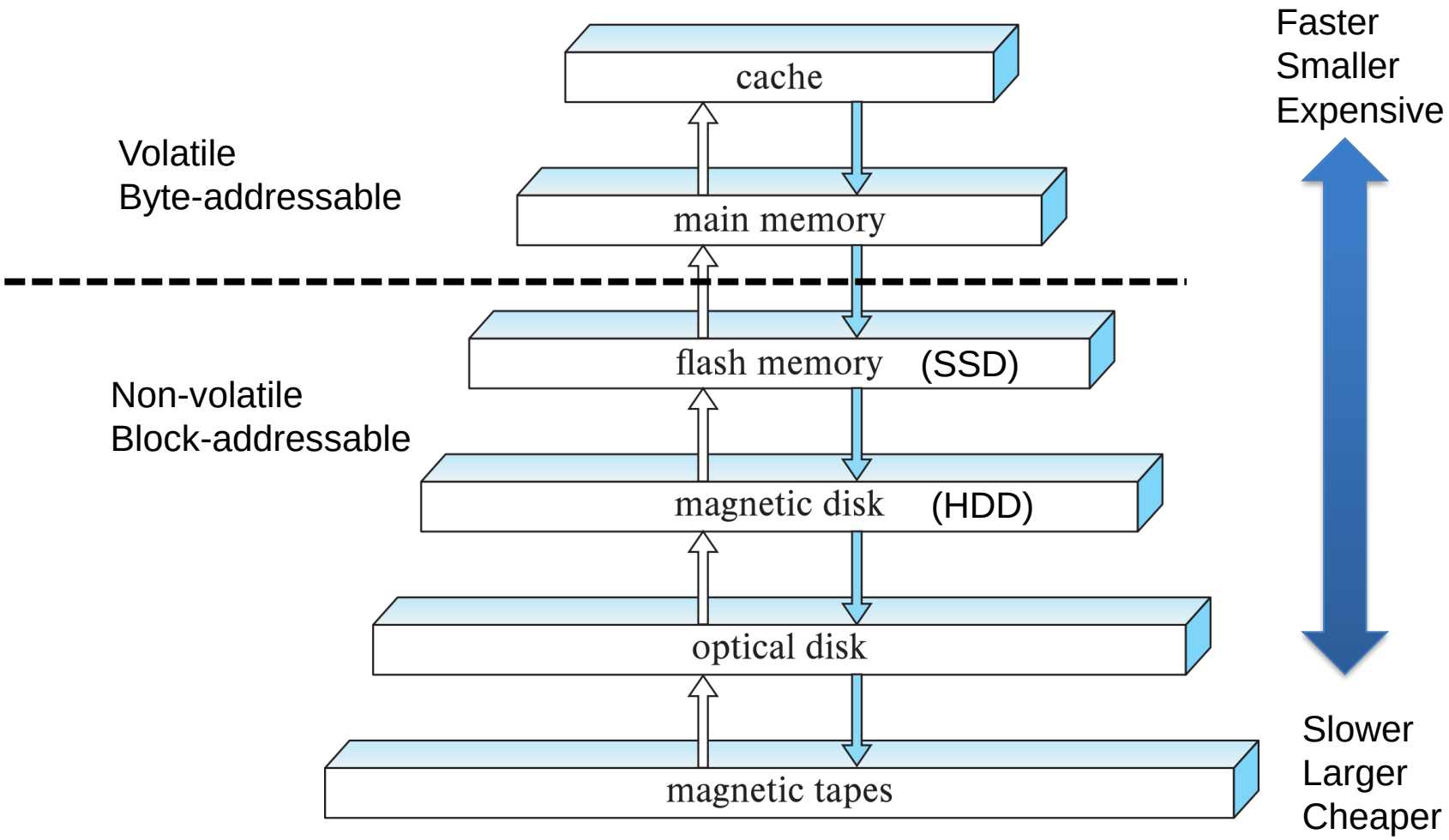
Data Storage

- Key Questions
  - How the DBMS represents the database in files on disk : Data Storage Structures
  - How the DBMS manages its memory and move data back-and-forth from disk: Data Storage Management

Overview of Physical Storage Media

- Can differentiate storage into:
  - volatile storage**:** loses contents when power is switched off
  - non-volatile storage:
    - Contents persist even when power is switched off.
    - Includes secondary and tertiary storage
- Factors affecting choice of storage media include
  - Speed with which data can be accessed
  - Cost per unit of data
  - Reliability

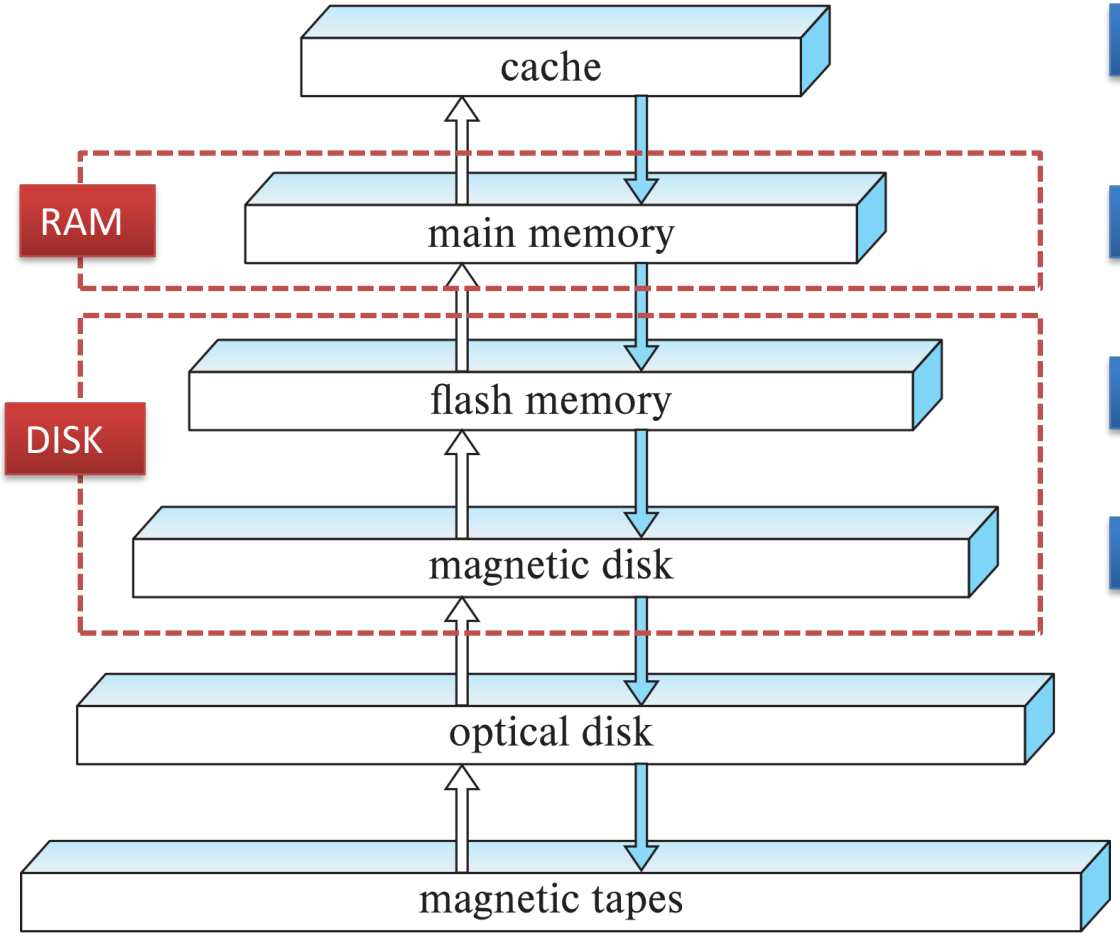
Storage Hierarchy



Storage Hierarchy (Cont.)

- primary storage: Fastest media but volatile (cache, main memory).
- secondary storage: next level in hierarchy, non-volatile, moderately fast access time
  - Also called **on-line storage**
  - E.g., flash memory, magnetic disks
- tertiary storage: lowest level in hierarchy, non-volatile, slow access time
  - also called **off-line storage** and used for **archival storage**
  - e.g., magnetic tape, optical storage
  - Magnetic tape
    - Sequential access, 1 to 12 TB capacity
    - A few drives with many tapes
    - Juke boxes with petabytes (1000's of TB) of storage

Typical Access Times



~0.000007 milliseconds

~0.0001 milliseconds

~0.02 to 0.15 milliseconds

~5 to 20 milliseconds

Data Storage Structures

- Databases – records
- Disks- blocks
  - disk block (contiguous sequence of bytes) is unit of storage
  - data are read or written in units of blocks

- Storage manager – pages
  - Size of page is chosen to be the size of a disk block
- Most databases use OS files as a intermediate layer for storing records.

Data Storage Structures

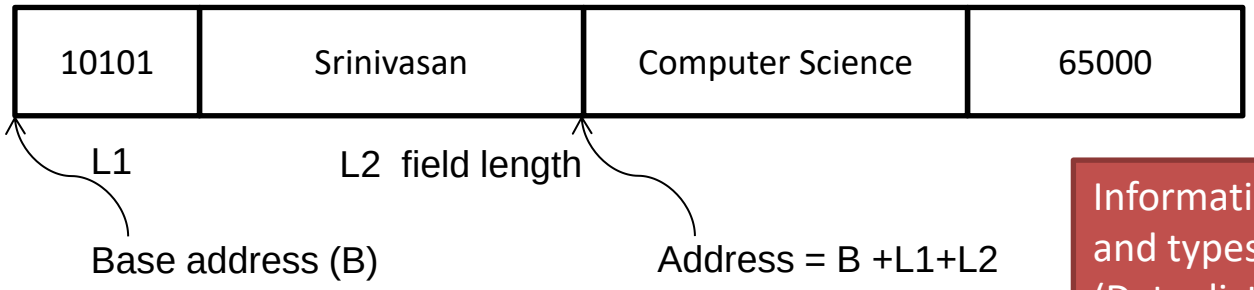
- The database is stored as a collection of *files*. Each file is a sequence of *records.* A record is a sequence of fields.

- Key questions
  - How do we organize fields in a record? – Record Layout
  - How do we organize tuples within a page? – Page Layout
  - How do we organize pages into a file? –File organization

Record Layout

- Record is identified using record id (*rid*)
- Records
  - Fixed length
  - Variable length
- Fixed-Length Records
  - each field has a fixed length
  - number of fields is fixed

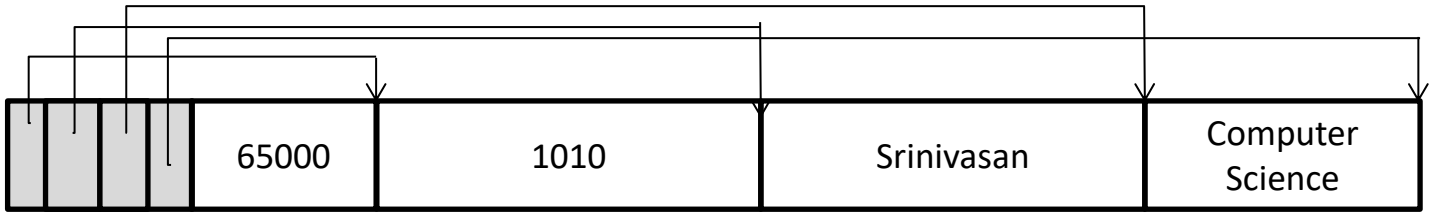
```
type instructor = record
        ID varchar (5);
        name varchar(20)
        dept_name varchar(20)
        salary numeric (8,2)
    end
```



Information about field lengths and types is in the system catalog (Data dictionary)

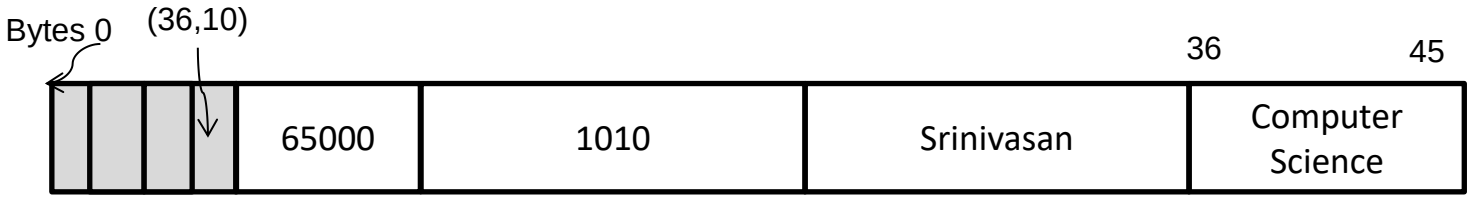
Record Layout

- Variable-Length Records
  - Variable length fields: strings (ID, name, dept)



Record Header:
(offset)

NULLS require no space



Record Header:
(offset, length)
Bitmap for null values

Record Layout

- Variable-Length Records Issues
  - Field may grow while modifying the record
  - Some DBMS (IBM DB2 and Microsoft SQL server) do not allow records to span pages whereas others (Oracle 8) allow large records to span pages and are organized as singly directed list.

Page Layout

- Issues to consider
  - 1 page = 1 disk block = fixed size (e.g. 8KB)
  - Records: Fixed length , Variable length

- One simple approach
  - Assume record size is fixed (later we will see for variable size record)
  - Each file has records of one particular type only
  - Different files are used for different relations

  (we assume that records are smaller than a disk block)

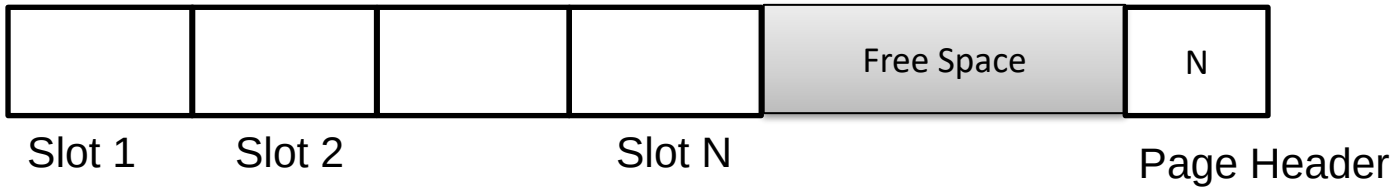
Page Layout

- Page - collection of slots
  - each slot contains a record

- Record is identified using record id (*rid*)
- Record id = RID
  - like a pointer to a tuple
  - Typically *rid* = <*pageID, slotNumber*>

- Fixed Length Records
  - uniform record slots arranged consecutively within a page

Page Layout



- How do we insert records?
- How do we delete records?

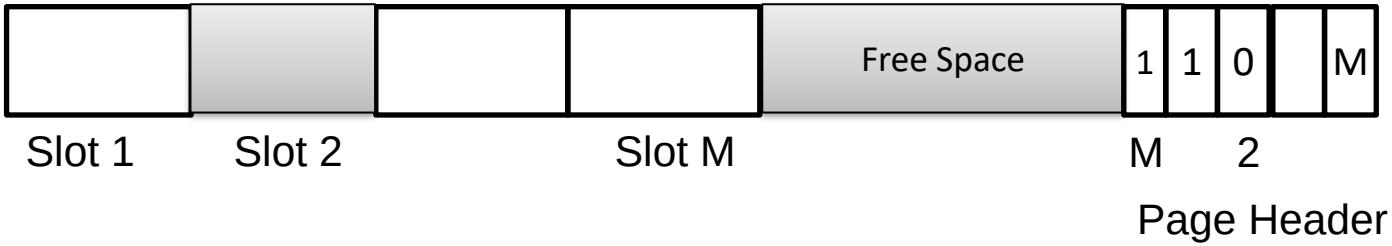
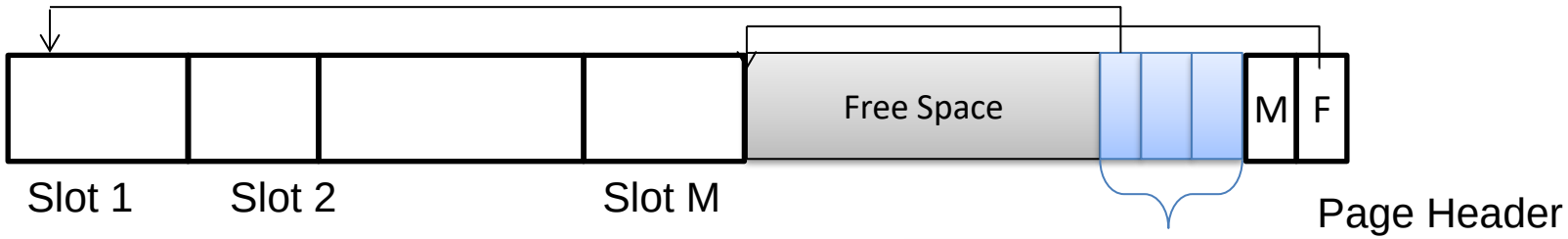


Figure: Slotted page organization for Fixed Length Records

Page Layout



Page header contains:
Slot directory + # entries + free space pointer

Slot Directory:
<record offset, record length>

Figure: Slotted page organization for Variable Length Records

- How do we insert records?
- How do we delete records?

File organization

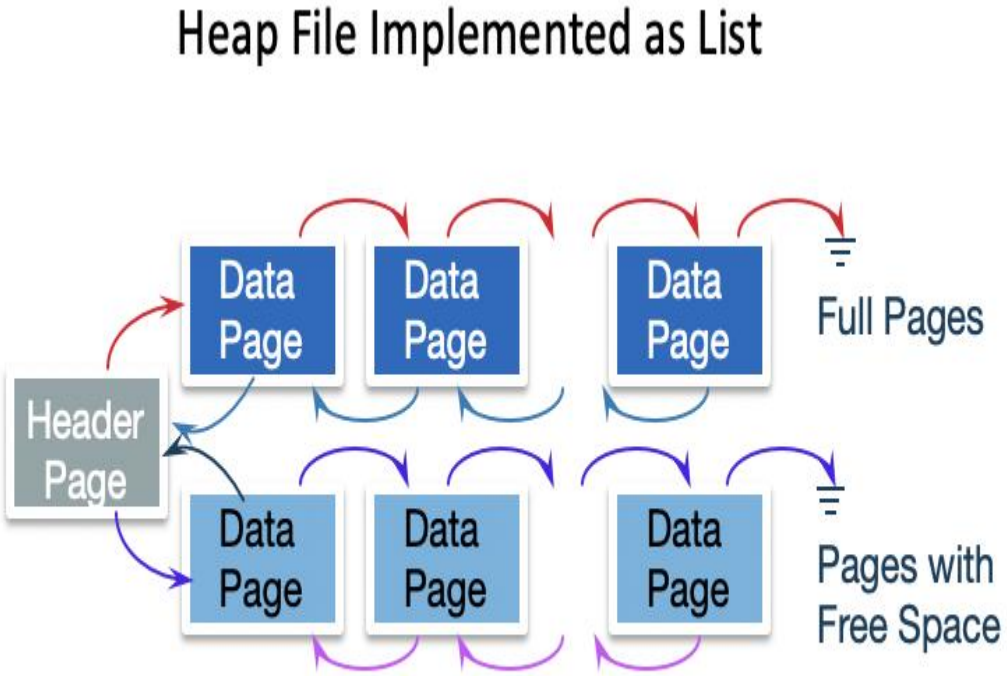
- How to organize collection of pages as file?

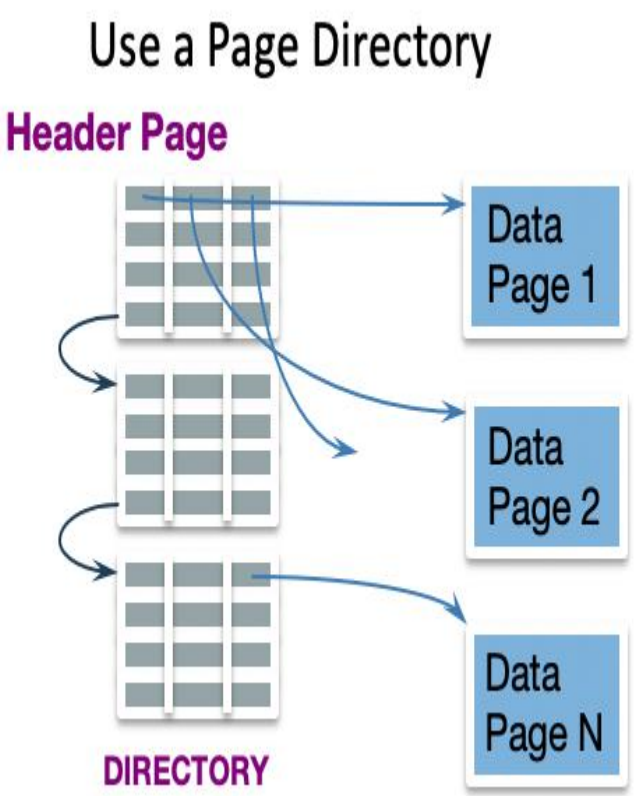
  (pages are collection of records, file can span several pages)

  - Heap file organization (unordered)
  - Sequential file organization (ordered)

- Heap file organization
  - records not ordered
  - **Insertion** can be done at the end of the file or utilize the free space created after record deletion.
  - need to keep track of free space

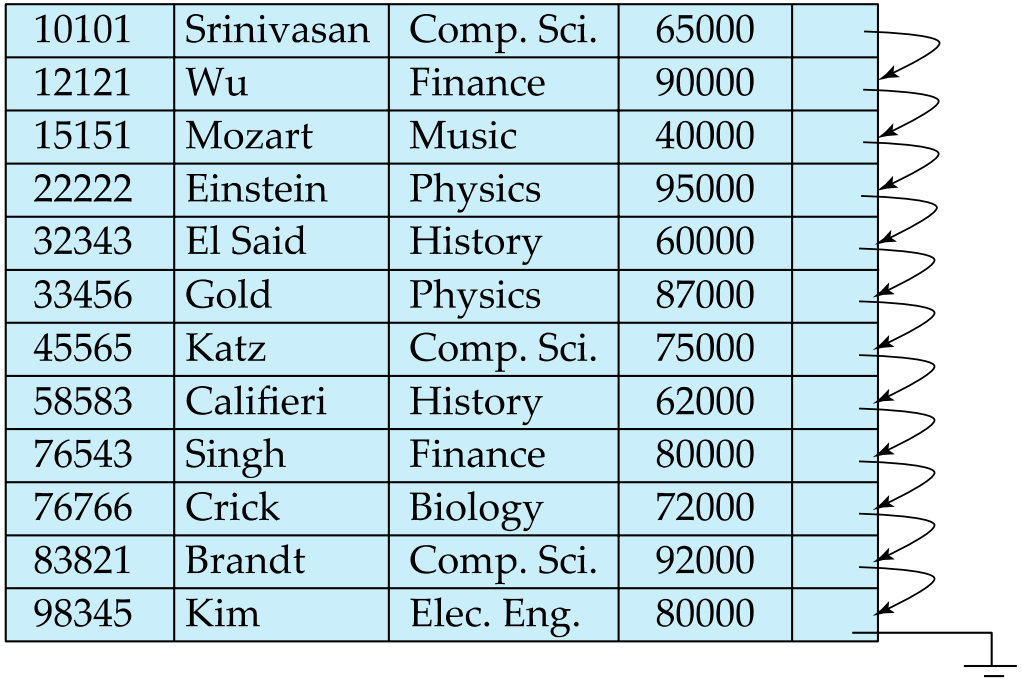
File organization





File organization

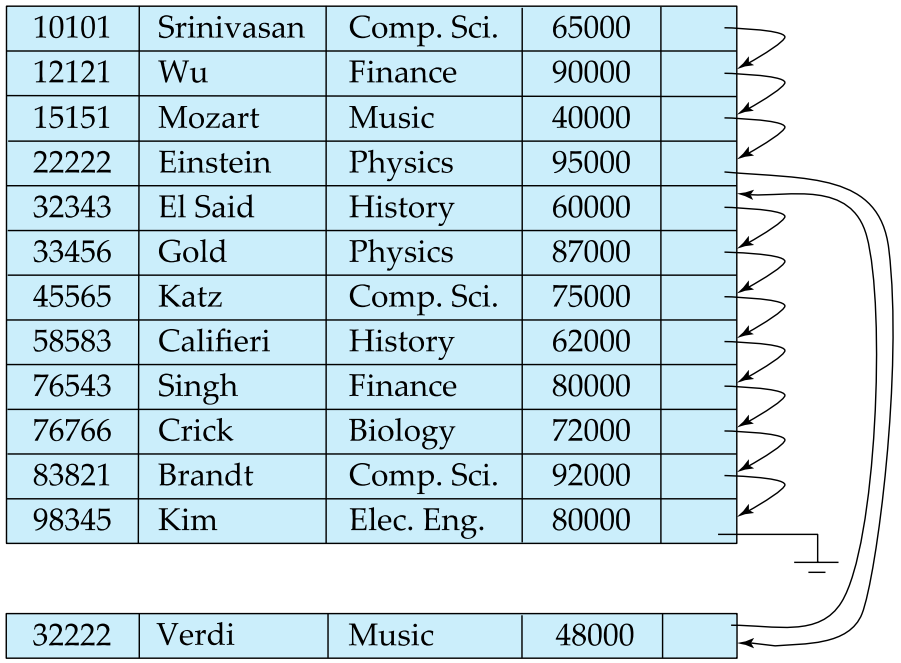
- Sequential file organization
  - records are sorted based on some search key
  - Search key – any attribute or set of attributes
  - each record points to the next in search key order



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

File organization

- Deletion – use pointer chains
- Insertion –locate the position where the record is to be inserted
  - if there is free space insert there
  - if no free space, insert the record in an overflow block
  - In either case, pointer chain must be updated
- Need to reorganize the file from time to time to restore sequential order

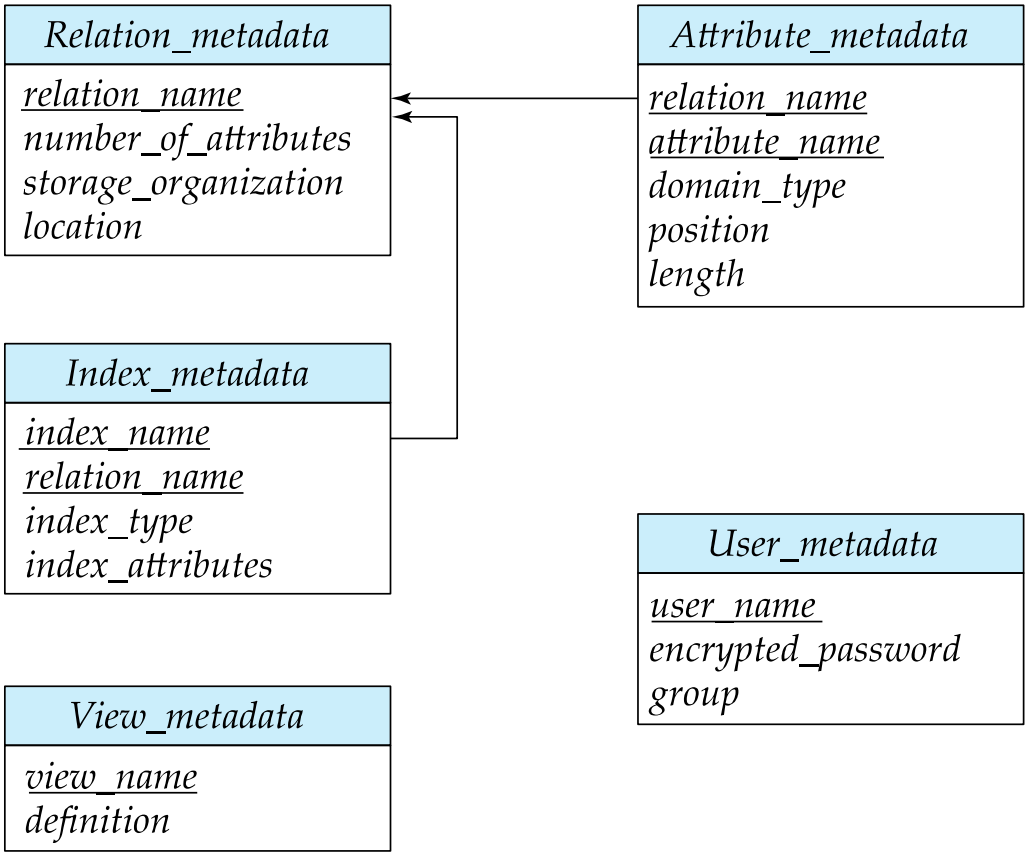


Data Dictionary Storage

- The Data dictionary (also called system catalog) stores metadata; that is, data about data, such as
- Information about relations
  - names of relations
  - names, types and lengths of attributes of each relation
  - names and definitions of views
  - integrity constraints
- User and accounting information, including passwords
- Statistical and descriptive data
  - number of tuples in each relation
- Physical file organization information
  - How relation is stored (sequential/heap/…)
  - Physical location of relation
- Information about indices (will be discussed in next lecture)

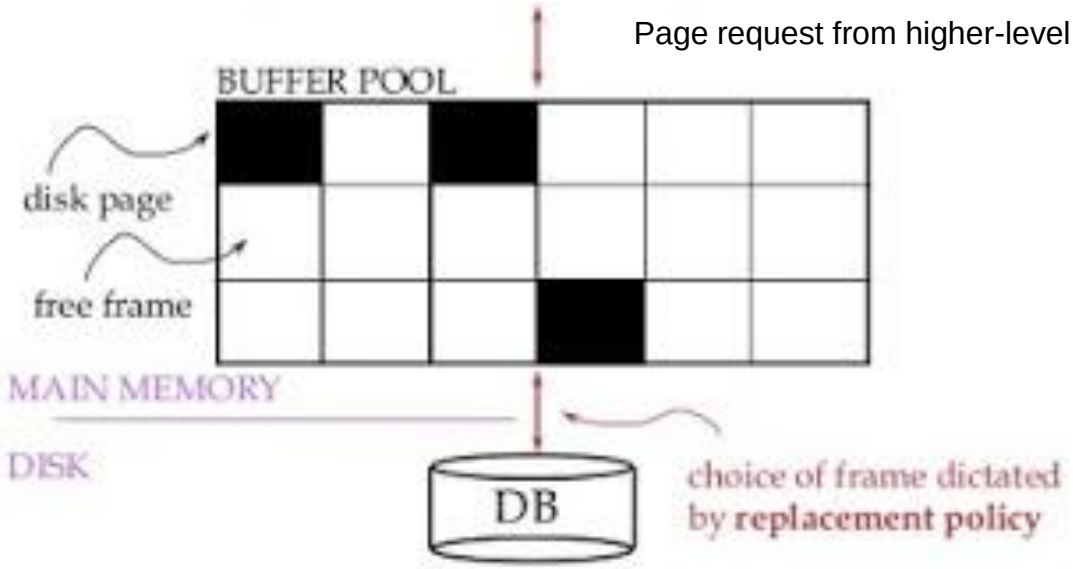
Relational Representation of System Metadata

- Relational representation on disk
- Specialized data structures designed for efficient access, in memory



Storage Access

- Blocks are units of both storage allocation and data transfer.
- Database system seeks to minimize the number of block transfers between the disk and memory. We can reduce the number of disk accesses by keeping as many blocks as possible in main memory.
- Buffer – portion of main memory available to store copies of disk blocks.
- Buffer manager – subsystem responsible for allocating buffer space in main memory.



Buffer Manager

- Programs call on the buffer manager when they need a block from disk.
  - If the block is already in the buffer, buffer manager returns the address of the block in main memory
  - If the block is not in the buffer, the buffer manager
    - Allocates space in the buffer for the block
      - Replacing (throwing out) some other block, if required, to make space for the new block.
      - Replaced block written back to disk only if it was modified since the most recent time that it was written to/fetched from the disk.
    - Reads the block from the disk to the buffer, and returns the address of the block in main memory to requester.

Buffer Manager

- Buffer replacement strategy
- Pinned block**:** memory block that is not allowed to be written back to disk
  - **Pin** done before reading/writing data from a block
  - **Unpin** done when read /write is complete
  - Multiple concurrent pin/unpin operations possible
    - Keep a pin count, buffer block can be evicted only if pin count = 0
- Shared and exclusive locks on buffer
  - Needed to prevent concurrent operations from reading page contents as they are moved/reorganized, and to ensure only one move/reorganize at a time
  - Readers get shared lock, updates to a block require exclusive lock
  - Locking rules:
    - Only one process can get exclusive lock at a time
    - Shared lock cannot be concurrently held with exclusive lock
    - Multiple processes may be given shared lock concurrently

Buffer-Replacement Policies

- Most operating systems replace the block least recently used (LRU strategy)
  - Idea behind LRU – use past pattern of block references as a predictor of future references
  - LRU can be bad for some queries
- Queries have well-defined access patterns (such as sequential scans), and a database system can use the information in a user's query to predict future references
- Mixed strategy with hints on replacement strategy provided by the query optimizer is preferable
- Example of bad access pattern for LRU: when computing the join of 2 relations r and s by a nested loops

  for each tuple *tr* of *r* do
    for each tuple *ts* of *s* do
      if the tuples *tr* and *ts* match …

```
select *
from instructor natural join
        department
```

Buffer-Replacement Policies (Cont.)

- Toss-immediate strategy – frees the space occupied by a block as soon as the final tuple of that block has been processed
- Most recently used (MRU) strategy – system must pin the block currently being processed. After the final tuple of that block has been processed, the block is unpinned, and it becomes the most recently used block.
- Buffer manager can use statistical information regarding the probability that a request will reference a particular relation
  - E.g., the data dictionary is frequently accessed. Heuristic: keep data-dictionary blocks in main memory buffer

Summary

- key question that we focused today
  - How the DBMS represents the database in files on disk : Data Storage Structures
    - Overview of storage hierarchy
    - Record layout
      - Fixed size
      - Variable size
    - Page layout
      - Slotted pages
    - File organization
      - Heap file organization
      - Sequential file organization

Summary

- key question that we focused today
  - How the DBMS manages its memory and move data back-and-forth from disk: Data Storage Management
    - Data Dictionary storage
    - Buffer manager
    - Buffer replacement policies
    - Pinning and unpinning blocks
    - Locks: shared and exclusive locks

Courtesy: some of the slides are take from lecture slides available at: https://www.db-book.com/ and http://pages.cs.wisc.edu/~dbbook/index.html